Generated PDF convenience snapshot
Authoritative source: `docs/guides/CODEBASE_GUIDE.md`
Source documentation baseline: `03f641a91564059d707be57304e6a1c18d09f783`
Generated: 2026-08-31
If this PDF and the Markdown source differ, the Markdown source controls.

Codebase Guide

Status: current implementation guide as of 2026-08-31.

Purpose

This guide explains how the Claude Code Backend Benchmark repository works as a research system.

It is intended for researchers and engineers who want to understand:

- how a benchmark request becomes a Harbor execution;
- how Claude Code is configured to talk to different model backends;
- how raw Terminal-Bench results are retained;
- how live observation differs from canonical publication;
- what Supabase and Cloudflare R2 each store;
- how reviewed evidence is created and protected;
- how the dashboard chooses between operational and reviewed data;
- how cost, validity, failure, and trajectory layers are kept separate;
- where important provenance boundaries are enforced;
- which code to inspect before making a particular kind of change.

This is deliberately a **concept-first** guide.

Do not begin by memorizing the directory tree.

Begin with the question:

> How does a research claim travel from benchmark configuration to retained evidence to reviewed interpretation to dashboard presentation?

The directory structure becomes much easier to understand after that flow is clear.

The repository's central design principle

The repository repeatedly chooses **separation over silent replacement**.

Important examples include:

- raw verifier outcome versus reviewed behavioral interpretation;
- live execution state versus canonical publication;
- Supabase metadata versus R2 artifact bytes;
- historical reviewed cost versus current provider-aware cost;
- provider evidence versus reviewed reconciliation authority;
- reconciliation authority versus promotion authorization;

- fixed reviewed comparison versus dynamic imported inventory;
- canonical identifiers versus friendly presentation labels;
- historical planning documents versus current operational state.

Many files that initially appear redundant exist because two pieces of evidence answer different questions.

Before removing a layer, merging two models, or replacing a historical file, identify which distinction that layer protects.

Architecture at a glance

The principal benchmark and evidence flow is:

```
research question
  -> phase / arm / task configuration
  -> local or GitHub Actions dispatch
  -> runner slot
  -> scripts/run_arm.sh
  -> scripts.lib.harbor
  -> Harbor
  -> Claude Code agent harness
  -> direct backend or LiteLLM-mediated backend
  -> Terminal-Bench task environment
  -> held-out verifier/tests
  -> Harbor result directory
  -> optional live supervision
  -> final canonical publication
  -> Supabase metadata + R2 artifact bytes
  -> reviewed snapshots / reconciliation layers
  -> dashboard loaders and models
  -> dashboard presentation
  -> research interpretation
```

There are several side paths:

```
live supervisor
  -> local redacted NDJSON
  -> optional live Supabase rows
  -> optional progressive R2 artifacts
```

and:

```
retained reviewed evidence
  -> offline review/classification generators
  -> manifest-bound checked-in snapshots
  -> file-backed dashboard loaders
```

and:

```
provider/account evidence
  -> normalized evidence source
  -> usage / pricing / cost evidence
  -> current usage reconciliation
  -> current cost reconciliation
  -> reviewed promotion gate where applicable
```

```
  -> benchmark.v_evidence_promotion_gate
  -> read-only Planner
```

and:

```
reviewed provider consistency
  -> checked-in current-reviewed reporting layer
  -> Overview / Cross-phase / Cost Coverage
```

These paths interact, but they are not interchangeable.

Seven layers to keep in mind

A useful code-reading model divides the repository into seven layers.

Layer 1 — Experimental design

Defines:

- benchmark phase;
- model/backend arm;
- task population;
- number of attempts;
- concurrency;
- route;
- agent harness.

Primary locations:

```
configs/phases/
configs/arms/
configs/tasks/
configs/router/
```

Layer 2 — Execution

Turns configuration into an actual Harbor invocation.

Primary locations:

```
scripts/run_arm.sh
scripts/lib/harbor.py
.github/workflows/phase3-arm-dispatch-v2.yml
```

Layer 3 — Raw evidence

Created by Harbor, Claude Code, and the verifier.

Primary locations:

```
results/
result.json
agent/claude-code.txt
```

```
trajectory.json
verifier/test-stdout.txt
verifier/ctrf.json
verifier/reward.txt
trial.log
exception.txt
```

Layer 4 — Operational publication

Makes completed and in-progress evidence queryable.

Primary locations:

```
scripts/run_arm_live.py
scripts/publish_phase3_run.py
scripts/ingest_phase3_run_metadata.py
scripts/lib/live_*.py
scripts/lib/canonical_publication.py
db/migrations/phase3/
```

Storage:

```
Supabase Postgres
Cloudflare R2
```

Layer 5 — Review, provider evidence, reconciliation, and promotion

Transforms retained benchmark/provider evidence into reproducible reviewed interpretations and explicit decision authority without changing raw outcomes.

Primary locations include:

```
scripts/provider_evidence/
scripts/collect_provider_evidence.py
scripts/review_evidence_promotion.py
scripts/generate_comprehensive_evidence_review.py
scripts/generate_failure_taxonomy_snapshot.py
scripts/generate_phase3_current_reviewed_comparison_v4.py
results/manual_verification/
results/phase3/reporting/
results/phase3/provider_usage/
db/migrations/phase3/011_provider_evidence_contract.sql
```

Layer 6 — Dashboard data/model layer

Reads operational database views or checked-in reviewed snapshots and converts them into explicit presentation models.

Primary location:

```
apps/dashboard/src/lib/
```

Layer 7 — Dashboard presentation

Presents research and evidence surfaces.

Primary location:

```
apps/dashboard/src/app/
apps/dashboard/src/components/
```

When modifying code, identify the layer first.

Experimental configuration

Phase configuration

Phase configuration lives in:

```
configs/phases/
```

The current Phase 3 router config is:

```
configs/phases/phase3-router.yaml
```

It defines, among other things:

- `phase_id`;
- dataset;
- agent;
- full task file;
- canary task file;
- smoke task file;
- default attempt count;
- default concurrency;
- result root;
- physical result subdirectories;
- planned arms.

The execution helper reads phase files through:

```
scripts/lib/arms.py
```

Important helpers include:

- `load_phase`;
- `task_file_for_mode`;
- `results_dir_for_mode`;
- `read_task_list`.

Historical-status warning

Some checked-in Phase 3 configuration text still describes Phase 3 as active.

That wording is retained historical configuration context, not evidence that Phase 3 should now be reopened.

Current-facing guides and runbooks explicitly mark Phase 3 closed; the historical configuration itself is intentionally preserved.

Do not infer current project phase from one old config description.

Arm configuration

Individual model routes live in:

```
configs/arms/
```

An arm config can define:

- canonical `arm_id`;
- display name;
- provider;
- agent harness;
- router;
- model alias presented to Harbor/Claude Code;
- backend model;
- expected observed model;
- result directory name;
- secret file;
- secret environment mapping;
- ordinary environment values;
- environment variables to clear;
- agent environment keys to forward;
- agent keyword arguments;
- notes/status.

For example, a routed OpenAI arm can instruct Claude Code to use an Anthropic- compatible LiteLLM endpoint while the actual backend is OpenAI.

That distinction is central:

```
Claude Code model alias
  != necessarily provider-native backend model
```

The route is part of the experimental condition.

The arm model already includes first-class `agent_harness` identity. The dashboard Arms surface exposes it when recorded.

This is the existing Phase 4 extension seam: future harness experiments should vary this canonical field rather than smuggling harness identity only into a display label. Harness **version** is not yet an equivalent first-class experiment contract and must be resolved before Phase 4 activation.

Canonical identity versus presentation

Do not casually rename `arm_id`.

Canonical identifiers participate in:

- result paths;
- database joins;
- reviewed snapshots;
- run selections;
- dashboard links;
- provider reconciliation;
- tests.

Friendly names belong in presentation layers.

A display-label change should normally not become a canonical-identity change.

Agent environment construction

`scripts/lib/harbor.py` constructs the child environment.

The basic sequence is:

1. copy the parent environment;
2. remove keys listed in `clear_env`;
3. read the configured ignored secret file;
4. map secret values into requested runtime environment names;
5. apply checked-in non-secret environment values;
6. apply per-run LiteLLM host/port override where appropriate;
7. pass only configured agent environment keys into Harbor.

This prevents a runner's ambient environment from silently defining the experiment.

The code also redacts sensitive `--agent-env` values before printing commands.

Why environment clearing matters

A routed arm may need to ensure that provider-native keys or unrelated Claude Code settings cannot accidentally change routing.

That is why arm files can explicitly clear variables such as provider API keys or agent-specific defaults.

Removing these clear lists without understanding the routing contract could make an arm behave differently while retaining the same arm ID.

Task selection

Task lists live under:

```
configs/tasks/
```

`scripts/lib/harbor.py` supports:

- phase-defined canary;
- phase-defined smoke;
- phase-defined full suite;
- explicit one-task diagnostic;
- explicit alternate task-file diagnostic.

Explicit task/task-file/ad-hoc labeling makes the run non-standard and places it under ad-hoc result storage.

Ad-hoc runs should not silently become scored full-suite evidence.

Attempt and concurrency rules

When not explicitly overridden:

- canary: one attempt;
- smoke: one attempt;
- ad-hoc: one attempt;
- full: phase default attempts;
- canary/smoke/ad-hoc concurrency: one;
- full: phase default concurrency.

These are execution defaults, not a claim that all historical runs used the same concurrency in every circumstance.

The actual Harbor command

`scripts/run_arm.sh` is intentionally small.

It delegates to:

```
python -m scripts.lib.harbor
```

`build_harbor_command` constructs a command equivalent in shape to:

```
uv run harbor run
  --dataset <dataset>
  --agent <agent>
  --model <model-if-configured>
  --agent-kwarg ...
  --agent-env ...
  --include-task-name ...
  --n-attempts ...
  --n-concurrent ...
  --jobs-dir ...
  --yes
```

Harbor is therefore the benchmark orchestrator.

The repository does not implement its own replacement Terminal-Bench scorer.

Agent harness versus backend

For Phases 1–3, Claude Code is the principal controlled agent harness.

The varied condition is primarily the model/backend route.

This is why Phase 4 is methodologically different: Phase 4 proposes changing the harness itself.

Do not treat a future harness comparison as merely another Phase 3 arm.

GitHub Actions execution envelope

The principal reviewed workflow is:

```
.github/workflows/phase3-arm-dispatch-v2.yml
```

It wraps the execution code with operational policy.

Inputs include:

- arm ID;
- canary/smoke/full mode;
- dry-run state;
- explicit paid-run confirmation;
- optional attempts/concurrency override;
- optional ad-hoc task/task-file;
- live-supervision switch;
- canonical-publication switch;
- progressive-artifact switch;
- Phase 3 repair authorization;
- runner label.

Six runner slots

The workflow exposes:

```
cc-bench-slot-1
cc-bench-slot-2
cc-bench-slot-3
cc-bench-slot-4
cc-bench-slot-5
cc-bench-slot-6
```

It also exposes broader pool/VPS labels.

The six-slot topology is an operational deployment detail layered on top of the benchmark configuration.

Do not encode a particular runner hostname into benchmark identity.

Paid-run safety

The workflow refuses a non-dry run unless:

```
confirm_paid_run = true
```

This is a human-intent safety boundary.

Do not remove it merely to make dispatch more convenient.

Future protected dashboard dispatch should preserve an equivalent explicit authorization boundary.

Canonical Phase 3 publication safety

The workflow defaults:

```
publish_results = false
```

for the closed Phase 3 environment.

A non-dry canonical publication into a completed Phase 3 suite additionally requires explicit repair authorization.

That is reinforced again inside the publisher.

Defense in depth is intentional.

Secret materialization

GitHub Actions secrets are converted at runtime into ignored `.secrets/*.env` files.

The files are given restrictive permissions.

Dry runs receive placeholders rather than real provider keys.

Paid runs receive only the provider secret relevant to the selected arm plus the router secret needed for the route.

Do not commit these runtime files.

Do not move secrets into arm YAML.

LiteLLM runtime

For routed Phase 3 arms, the workflow:

1. copies the checked-in LiteLLM example to a runtime config path;
2. verifies the requested route alias exists;
3. installs/uses the isolated LiteLLM proxy environment when needed;
4. starts/ensures the proxy;
5. starts arm-specific supporting services when needed.

The currently installed LiteLLM version is runtime state.

It should not be retroactively treated as the historical version of an old benchmark run.

Runtime provenance

The workflow records runtime provenance before the benchmark.

`scripts/lib/runtime_versions.py` captures:

- Harbor distribution version;
- LiteLLM version from the isolated proxy Python environment;
- runner-side Claude Code CLI version.

At final publication it also inspects retained:

```
agent/claude-code.txt
```

for observable Claude Code `system/init` records.

The observed transcript parser is intentionally bounded.

It does not search an unbounded transcript for an arbitrary version string.

Runtime version versus publication identity

Runtime provenance is stored with the canonical manifest.

However:

```
scripts/lib/publication_fingerprint.py
```

does not include `runtime_versions` in its `RUN_FIELDS`.

That means runtime provenance can enrich the historical record without silently changing the benchmark publication identity contract.

Do not add fields to the fingerprint casually.

A fingerprint change is an identity-policy change.

Result directories

Harbor produces result directories below the jobs directory selected by the phase/arm/mode configuration.

For Phase 3:

- canary has its own subdirectory;
- smoke has its own subdirectory;
- full runs are physically stored under `raw`;
- ad-hoc evidence has separate ad-hoc paths.

Logical mode and physical storage mode are therefore distinct concepts.

The ingestion code explicitly handles that distinction.

Raw benchmark authority

The authoritative correctness signal remains the task verifier/reward retained by Harbor.

Typical evidence includes:

```
result.json
agent/claude-code.txt
trajectory.json
verifier/test-stdout.txt
verifier/ctrf.json
verifier/reward.txt
trial.log
exception.txt
config.json
```

Artifact availability varies by trial.

Do not assume every trial has every optional artifact.

Artifact typing

`scripts/ingest_phase3_run_metadata.py` maps known retained filenames to artifact types such as:

- result;
- trajectory;

- log;
- agent transcript;
- configuration;
- verifier CTRF;
- verifier stdout;
- verifier reward;
- exception.

Only allowlisted artifact names participate in canonical artifact collection.

That protects the publisher from recursively uploading arbitrary runner files.

Artifact hashing

Canonical artifacts record:

- local path;
- SHA-256;
- byte size;
- R2 key/URI where applicable.

The hash is part of evidence identity.

A path by itself is not enough to establish artifact identity.

R2 key construction

Canonical R2 object keys incorporate evidence identity including:

- prefix;
- phase;
- mode;
- arm;
- execution scope when applicable;
- run timestamp;
- content hash;
- relative path.

This allows object verification and reduces the chance that two executions silently overwrite one another.

Live supervision is a sidecar

The optional live supervisor is:

```
scripts/run_arm_live.py
```

It wraps the benchmark command.

It does **not** replace:

- Harbor;

- Terminal-Bench;
- the verifier;
- final canonical publication.

Its role is observation.

What live supervision observes

It can retain:

- process stdout/stderr;
- warnings;
- heartbeats;
- elapsed time;
- completed-trial evidence;
- partial aggregate counters;
- stable artifacts;
- publication warnings.

Observable process output is not labeled as private model reasoning.

Local-first live evidence

`scripts/lib/live_events.py` writes redacted local NDJSON before offering an event to a shared sink.

The sequence is conceptually:

```
observed event
  -> redact
  -> append local NDJSON
  -> update local latest index
  -> optionally publish shared event
```

This is an important failure-isolation property.

A Supabase outage should not erase the local execution-observation record.

Redaction

`Redactor` protects shared/live text using both:

- known runtime secret values;
- pattern-based detection.

It covers common forms of:

- bearer tokens;
- provider keys;
- database URLs;
- access keys;
- token/secret/password assignments.

It also reads ignored runtime secret env files as redaction sources.

Redaction is a defense, not permission to print secrets intentionally.

Bounded process output

Live process output uses bounded queues and shared-output sampling.

The design avoids allowing unlimited process output to become unlimited shared database state.

Warning/error-like output is handled separately from routine sampled output.

Deterministic live-run identity

A live execution ID is derived from:

- GitHub run ID;
- GitHub run attempt;
- runner name;
- arm ID;
- mode;
- a digest.

The runner display name participates in this live execution identifier, but does not become the canonical scientific identity of the benchmark arm.

Live database schema

Migration:

```
db/migrations/phase3/009_live_run_supervision.sql
```

adds:

```
benchmark.live_runs
benchmark.live_run_events
benchmark.live_trials
benchmark.live_artifacts
```

These tables are explicitly non-canonical.

The live run contains an optional:

```
canonical_arm_run_id
```

which is populated only when final publication succeeds.

There is not a direct canonical foreign key for every live trial/artifact row.

Reconciliation is a publication process, not a row-for-row identity assumption.

Live-state monotonicity

`scripts/lib/live_db.py` contains explicit merge rules.

Examples:

- completed trial evidence should not regress to an earlier partial state;
- token/cost/runtime counters are monotonic where appropriate;
- artifact stability advances rather than regresses;

- terminal run/publication states are protected;
- event insertion is idempotent by sequence.

These rules matter because live observation can arrive repeatedly or out of order.

Live publication failure

The live database publisher can spool failed shared publications locally.

That makes shared live state useful but non-authoritative.

Final benchmark execution remains determined by the wrapped child process.

Progressive artifacts

When enabled, stable completed-trial artifacts may be uploaded while the run is still executing.

Progressive upload is optional.

Final canonical publication does not require that progressive publication already succeeded.

The final publisher can upload missing canonical objects.

Why progressive publication requires stronger controls

The workflow requires progressive artifacts on a paid run to be coupled with:

- live supervision;
- final publication.

This avoids creating an accidental partial remote evidence stream with no intended canonical completion path.

Canonical publication is separate

Final publication is performed by:

```
scripts/publish_phase3_run.py
```

This is an after-execution operation.

It can run even if live supervision was disabled.

The key conceptual boundary is:

```
live observation != canonical publication
```

Publication discovery context

Before execution, the workflow records:

- watch roots;
- baseline run directories;
- expected trial count;
- execution identity;
- runtime versions;
- start epoch.

Final publication uses that context to identify the run directory created by the current execution instead of blindly selecting the newest directory.

That is a major provenance safeguard.

Workspace path containment

`scripts/lib/path_safety.py` enforces filesystem boundaries.

It rejects or constrains:

- paths outside the supplied workspace;
- symlink escapes;
- output paths with unsafe parents;
- artifact traversal;
- files outside approved roots.

This applies to live state, publication state, manifests, artifacts, and result discovery.

Do not replace these helpers with unbounded `Path.resolve()` calls without preserving their policy.

Canonical eligibility

Before publication, `scripts/lib/publication_eligibility.py` checks the final run structure.

Among other things it verifies:

- final root `result.json` is parseable;
- final timestamps are present and ordered;
- declared total trials are valid;
- completed/error statistics exist;
- running/pending/cancelled counts are zero;
- every discovered trial has final evidence;
- discovered trial count matches expected count where supplied;
- discovered count matches root total;
- root completed count matches root total;
- discovered exception count matches root error count.

A failed benchmark can still be structurally complete enough to publish.

An interrupted or partial benchmark is not automatically canonical merely because some trials exist.

Manifest construction

`build_manifest` in:

```
scripts/ingest_phase3_run_metadata.py
```

turns the retained result directory into canonical metadata.

The manifest represents:

- run;
- arm;
- trials;

- artifacts;
- hashes;
- costs/tokens;
- logical/storage mode;
- suite;
- Git/GitHub execution context.

Final publication then adds runtime provenance and publication state.

Publication fingerprint

`scripts/lib/publication_fingerprint.py` builds a deterministic SHA-256 over selected canonical facts.

Its payload includes selected run fields plus normalized trial and artifact records.

Artifact fingerprint records include:

- artifact type;
- normalized run-relative path;
- SHA-256;
- size.

The fingerprint is used to distinguish an idempotent replay from a conflicting publication.

It is not intended to hash every incidental metadata field.

Local integrity before remote mutation

Before uploading or inserting canonical metadata, the publisher verifies local artifact hashes and sizes against the manifest.

It verifies them again before database insertion.

This protects against the retained run directory changing during publication.

R2 before canonical database commit

For normal full publication, the order is intentionally:

1. build manifest;
2. verify local artifacts;
3. reconcile existing/progressive evidence where applicable;
4. upload missing R2 objects;
5. verify R2 checksum/size/object integrity;
6. insert canonical database metadata;
7. verify canonical database relationships;
8. commit.

This prevents a successfully committed canonical row set from claiming remote artifacts that were never verified.

Transactional canonical publication

The transaction layer is:

```
scripts/lib/canonical_publication.py
```

It keeps:

- canonical insertion;
- live-to-canonical transition;
- relationship verification

inside one database transaction.

If verification fails:

```
rollback
```

is required.

This is why publication verification code should not be converted into a post-commit warning.

Publication concurrency protection

When no live-run identity is available, canonical publication obtains a transaction advisory lock over the publication identity.

This reduces race conditions between concurrent attempts to publish the same canonical identity.

Replay behavior

A repeated final publication can be accepted as:

```
already_completed
```

only after the existing publication matches the expected fingerprint and, where required, R2 integrity is verified.

Idempotency means:

> repeat the same publication safely

not:

> overwrite whatever already exists.

Closed Phase 3 suites

`scripts/lib/phase3_freeze.py` protects:

```
phase3-canary-1
phase3-smoke-5
phase3-full-20
```

A new non-dry canonical publication into those suites is refused unless an explicit repair authorization is supplied.

This is a scientific freeze boundary.

Do not remove it merely because future work needs new benchmark execution.

Future phases should use new identities/suites.

Canonical Supabase schema

The initial canonical schema begins in:

```
db/migrations/phase3/001_benchmark_metadata.sql
```

Core entities include:

```
benchmark.benchmark_runs
benchmark.benchmark_arms
benchmark.benchmark_tasks
benchmark.benchmark_trials
benchmark.benchmark_artifacts
```

Additional migrations add:

- dashboard views;
- idempotent-ingestion constraints;
- cost coverage;
- eval suites and arm runs;
- quality flags;
- valid-only views;
- adjusted cost coverage;
- live supervision;
- cost-authority semantics;
- normalized provider evidence, pricing snapshots, and usage/cost reconciliations;
- durable promotion-gate history and the fail-closed promotion view.

Important current migrations include:

```
db/migrations/phase3/010_cost_authority_semantics.sql
db/migrations/phase3/011_provider_evidence_contract.sql
```

The migrations are historical schema evolution.

Do not edit an already-applied migration to represent a new schema change.

Add a new migration.

Supabase versus R2

The division is:

Supabase/Postgres

Stores queryable:

- run metadata;
- arm metadata;
- task metadata;
- trial metadata;

- cost/token/runtime values;
- validity;
- relationships;
- artifact metadata;
- live state;
- normalized provider evidence sources;
- provider usage and cost evidence;
- provider pricing snapshots;
- current/historical usage and cost reconciliations;
- durable promotion-review history;
- fail-closed promotion state;
- derived/query views.

Cloudflare R2

Stores large evidence bytes:

- result JSON;
- logs;
- transcripts;
- trajectories;
- verifier outputs;
- configuration artifacts;
- other allowlisted retained evidence.

A Postgres `r2_uri` is a reference.

It is not proof that the bytes were retrieved or verified by a particular dashboard render.

Dashboard database access

Operational dashboard queries enter through:

```
apps/dashboard/src/lib/db.ts
```

The dashboard reads:

```
SUPABASE_DB_URL
```

server-side and uses a Postgres connection pool.

The shared `queryRows()` boundary acquires a pool client, begins an explicit read-only transaction, executes the requested read, commits on success, and rolls back on failure before releasing the client. Operational dashboard code should not bypass that boundary with direct pool queries. This is an application-level defense in addition to database permissions; it does not turn the dashboard into a database writer.

Database credentials do not belong in client components.

Most operational query functions are implemented in:

```
apps/dashboard/src/lib/dashboard-data.ts
```

This file is a major place to inspect when a displayed operational denominator or filter is surprising.

Database views are not raw truth replacements

SQL views provide useful:

- aggregation;
- validity filtering;
- cost coverage;
- quality summaries;
- dashboard shape.

They derive from canonical rows.

They do not change the original verifier result.

Operational dashboard populations

Operational queries can represent:

- all imported;
- valid imported;
- a selected suite;
- an exact run;
- an exact trial;
- an exact artifact;
- current live state.

Always inspect the SQL or scope selector before assuming that two operational pages share a denominator.

Checked-in reviewed evidence

Some of the dashboard's most important research views deliberately do **not** derive their headline numbers from the latest database rows.

They use checked-in reviewed snapshots.

This protects reproducibility.

Important reviewed sources include:

```
results/manual_verification/comprehensive_review_20260731/
results/manual_verification/failure_taxonomy_20260813/
results/phase3/reporting/phase3_extended_reviewed_comparison_20260805.json
results/phase3/reporting/phase3_current_reviewed_comparison_20260821.json
results/phase3/reporting/phase3_reviewed_run_selection_20260809.json
```

Comprehensive-review loader

Dashboard loading for the comprehensive review is implemented in:

```
apps/dashboard/src/lib/review-data.ts
```

This is not a casual CSV reader.

It validates:

- expected manifest schema;
- analyzer version;
- exact output filename whitelist;
- file byte bounds;
- per-output SHA-256;
- per-output byte count;
- source analyzer hash;
- source generator hash;
- row counts.

If those contracts fail, the loader exposes states such as:

- unavailable;
- stale;
- mixed output.

It does not silently mix old and new review files.

Why manifest binding matters

Imagine someone manually edits:

```
trial_review.csv
```

but not the manifest.

The loader should reject the resulting mixed snapshot.

Imagine someone regenerates outputs with changed analyzer code but leaves old metadata.

The loader should detect source-hash disagreement.

The point is reproducibility, not distrust of CSV.

Raw outcome remains preserved

The comprehensive review adds interpretations such as:

- execution validity;
- activity subtype;
- policy disposition;
- failure subtype;
- termination subtype;
- telemetry status;
- confidence;
- evidence completeness.

It does not replace the raw outcome.

That rule is visible both in the data model and in trial-detail presentation.

J2 failure taxonomy

The second-stage failure/trajectory layer is rooted in:

```
configs/dashboard/failure_taxonomy_v1.json
```

and generated output under:

```
results/manual_verification/failure_taxonomy_20260813/
```

The registry defines independent axes such as:

- response path;
- verifier failure;
- assertion failure;
- trajectory disposition.

The ordering fields in the registry are display order.

They are not hidden classifier precedence.

Why independent axes matter

A trial can simultaneously be:

- a raw success;
- timeout-affected;
- a substantive execution;
- not a verifier failure.

Or:

- a raw failure;
- provider-policy blocked;
- not meaningfully classifiable as a wrong solution.

Code that collapses all of these to one `failure_reason` would discard useful research information.

DR-302 failure composition

DR-302 lives in dashboard model code such as:

```
apps/dashboard/src/lib/failure-composition.ts
```

It creates one mutually exclusive display category for each raw failure.

That is a **presentation partition**.

It does not redefine the underlying J2 axes.

When modifying DR-302, inspect:

```
apps/dashboard/src/lib/failure-composition.test.mjs
```

```
apps/dashboard/src/components/FailureCompositionPanel.test.mjs
```

The tests protect:

- exact frozen source membership;
- global counts;
- per-arm partitioning;
- explicit display precedence;
- separation of successful timeout anomalies.

Historical DR-303 spend decomposition

Historical spend decomposition is implemented through:

```
apps/dashboard/src/lib/spend-decomposition-source.ts
apps/dashboard/src/lib/spend-decomposition.ts
```

It preserves the historical reviewed cost model.

DR-303 answers questions about the historical Phase 3 accounting layer.

It is not a generic receptacle for every later provider-cost correction.

Historical DR-304 milestone and current V4 reviewed layer

DR-304 first introduced an **additive** provider-aware current selected-cost model, including exact selected-run OpenAI provider billing without rewriting the historical reviewed accounting layer.

The earlier 2026-08-21 DR-304 snapshot is retained historical provenance. It is not the current decision-facing selected-cost source.

The current implementation is V4.

Generator:

```
scripts/generate_phase3_current_reviewed_comparison_v4.py
```

Canonical generated JSON:

```
results/phase3/reporting/phase3_current_reviewed_comparison_20260825.json
```

Dashboard generated mirror:

```
apps/dashboard/src/generated/phase3-current-reviewed-comparison-data-v4.ts
```

Dashboard validator/loader:

```
apps/dashboard/src/lib/phase3-current-reviewed-comparison.ts
```

The V4 generator consumes pinned:

- frozen historical reviewed comparison;
- 2026-08-25 current arm cost reconciliation;
- 2026-08-25 provider cost evidence matrix;

- supporting Anthropic exception lower-bound reconciliation.

It never rewrites the frozen historical reviewed comparison.

All 16 reviewed extended arms have V4 **reporting reconciliation rows**. Do not confuse those reporting rows with Migration-011 normalized reconciliation chains: the later cross-provider consistency contract records 10 normalized selected arms and 6 explicit accepted normalized-absence arms.

Current V4 fail-closed behavior

The generator/loader encode expected:

- source paths, roles, and hashes;
- schemas and generator identity;
- exact core/extended scope membership;
- reviewed run identities;
- provider/evidence state;
- selected cost basis and relation;
- allocation states;
- selected scope totals.

If those facts change unexpectedly, generation/loading should fail.

This is intentional friction.

A current-cost model change is a research-methodology change.

Current cross-provider reviewed evidence layer

DR-304 was an important intermediate step, but it is no longer the whole current provider-aware story.

The final selected-run consistency contract is documented in:

```
docs/reports/phase3/PHASE3_CROSS_PROVIDER_CONSISTENCY_20260828.md
```

For the 16 reviewed selected Phase 3 arms it records:

```
8 provider families
10 normalized current reconciliation chains
6 accepted normalized-absence arms
```

Normalized authority classes are:

```
2 exact provider-billed
5 qualified rate estimates
3 qualified lower bounds
```

Accepted normalized absence is separate:

```
4 Anthropic
2 GLM
```

Anthropic accepted absence means that no retained first-party selected-run provider source was available for normalization under the reviewed evidence/credential set. It is not a claim that Anthropic lacks provider APIs.

GLM accepted absence is deliberate for two distinct evidence limits: historical GLM 5.1 provider context was not allocable to its selected run, while comparable selected-run first-party evidence was not retained for GLM 5.2.

The checked-in current reviewed reporting layer remains decision-facing even when normalized provider rows are deliberately absent. Do not fabricate provider evidence merely to make every arm structurally identical.

Provider evidence schema and promotion review

Migration 011 adds first-class relations for:

```
benchmark_provider_evidence_sources
benchmark_provider_usage_evidence
benchmark_provider_pricing_snapshots
benchmark_provider_cost_evidence
benchmark_usage_reconciliations
benchmark_usage_reconciliation_sources
benchmark_cost_reconciliations
benchmark_cost_reconciliation_sources
benchmark_evidence_promotion_gates
```

and the derived view:

```
benchmark.v_evidence_promotion_gate
```

Raw harness values remain provenance. Reconciliations select the current decision-facing usage/cost authority instead of overwriting those raw values.

Promotion review is another distinct layer.

The durable operator:

```
scripts/review_evidence_promotion.py
```

supports:

```
--plan
--check-only
--rollback-only
--apply
```

Mutation modes require exact usage/cost/current-gate pins plus a state SHA-256 from the preceding read-only check. They take an advisory transaction lock, re-read the state, preserve superseded gate history, and verify the fail-closed view.

A `waived` decision is durable provenance but is deliberately non-authorizing.

For the operator workflow use:

```
docs/runbooks/EVIDENCE_PROMOTION_REVIEW.md
```

Current versus historical cost consumers

The dashboard has code that intentionally chooses which pages use:

- current selected cost;

- historical reviewed cost;
- recorded operational cost;
- historical DR-303 outcome allocation.

Do not perform a repository-wide search/replace of one cost field with another.

First determine what question the page is answering.

Allocation firewall

For provider-billed OpenAI totals:

- arm-level total is available;
- trial-level provider allocation is `unavailable_provider_aggregate`;
- outcome-level provider allocation is `unavailable_provider_aggregate`.

That enum means the exact provider total is known only at the selected arm/run aggregate level. It must not be fabricated into per-trial or per-outcome dollars.

Therefore current selected total must not inherit historical outcome shares.

This is enforced in current-reviewed loaders/models/tests.

The fact that a ratio can be calculated from an arm total does not create a trial allocation.

Generated dashboard modules

Some checked-in JSON snapshots have generated TypeScript mirrors so dashboard code can consume them deterministically.

Tests compare generated modules with canonical JSON.

Do not hand-edit only the generated TypeScript mirror.

Identify the canonical source and generator first.

Reviewed run selection

The reviewed Phase 3 comparison uses a frozen exact run-selection snapshot.

This is why reviewed dashboard links do not select:

```
newest run for arm
```

at render time.

Exact run identity is part of the review provenance.

Changing selected reviewed runs requires a review-layer change, not a generic database query tweak.

Dashboard layers

A useful way to read the Next.js app is:

`src/app`

Routes and page composition.

`src/components`

Reusable presentation and interactive components.

`src/lib`

Data access, reviewed loaders, evidence contracts, pure models, formatting, scope selection, provenance, and safety helpers.

`src/generated`

Generated reviewed snapshot modules.

Do not assume every file under `src/lib` is safe for client import.

Some are deliberately server-only or depend on filesystem/database access.

Server versus client boundaries

Most evidence and database loading should remain server-side.

Client components are used where interaction is necessary, such as the cost/performance chart.

The chart's client component consumes a pure prepared data/view layer rather than importing server database or filesystem code.

This boundary reduces accidental credential or server-only dependency leakage.

Dashboard Overview

Overview combines explicitly separated sources.

Its reviewed comparison uses checked-in reviewed evidence and exact selected runs.

Other inventory/discovery sections can use dynamic database evidence.

This is intentional.

Do not "simplify" Overview by forcing everything through one universal source.

Trial detail

Trial detail is a convergence point.

It can combine:

- canonical trial metadata;
- artifact metadata;
- frozen comprehensive review;
- frozen J2 taxonomy;
- optional bounded current artifact analysis;
- task instruction context.

The page labels the source of diagnosis and evidence.

When a validated snapshot exists, current artifact analysis does not silently replace it.

Artifact detail

Artifact pages combine:

- canonical metadata;
- object provenance;
- bounded R2/local preview;
- related artifacts;
- trial/run/task links.

The content reader distinguishes:

- indexed;
- retrieved;
- complete;
- bounded;
- verified.

Do not collapse those states to a single boolean such as `has_artifact`.

Provider Evidence

The Provider Evidence routes are:

```
/provider-evidence
/provider-evidence/[sourceId]
```

They are server-rendered, source-centric consumers of the normalized Migration-011 provider evidence and reconciliation tables.

The index pages provider source records before expanding linked run/arm and reconciliation context, avoiding child-row fan-out as a pagination mechanism. Source detail loads normalized usage, cost, pricing snapshots, usage/cost reconciliations, every reconciliation source role, and the pricing source independently. That last relationship matters because the pricing snapshot can belong to a different evidence source than the source being viewed.

The route deliberately does not select or render arbitrary `raw_metadata`. Displayed URIs, notes, provider references, labels, and structured pricing content pass through the existing sanitization/redaction boundary. The browser does not call provider APIs and does not write provider evidence or reconciliations.

Architecture and Data Model routes

These routes are code-adjacent documentation:

```
/architecture
/data-model
/glossary
```

They are useful places to update when architectural behavior changes.

The Data Model route is not a live schema inspector.

It documents checked-in relationships and authorities.

Planner

Planner reads checked-in:

```
configs/arms/
configs/tasks/
```

and produces reviewable planning material.

It is intentionally read-only.

For Smoke and Full it also reads the current predecessor evidence review from:

```
benchmark.v_evidence_promotion_gate
```

The Planner displays the exact source arm run, reconciliation identities, selected usage/cost authority, reviewer, limitations, blockers, and waiver state used for the current gate. When effective advancement is absent it withholds the corresponding commands.

The Planner does **not** write promotion gates.

Durable review is performed separately by:

```
scripts/review_evidence_promotion.py
```

The Planner also does not dispatch GitHub Actions.

Future protected dashboard dispatch should therefore be implemented as a new controlled capability rather than by quietly adding a browser-side token to the existing client.

Before the first paid Phase 4 Canary, promotion-gate scoping must be reviewed because the current unique/current slot is arm + target mode. A future harness experiment should not inherit authorization from a different experiment merely because an arm identifier is reused.

Evidence links

Evidence-link helpers preserve source/scope context when navigating from a reviewed aggregate into exact operational evidence.

Changing a link can therefore change provenance semantics.

When modifying evidence navigation, inspect:

```
apps/dashboard/src/lib/evidence-links.ts
apps/dashboard/src/lib/evidence-links.test.mjs
apps/dashboard/src/lib/evidence-source-wiring.test.mjs
```

Presentation labels

Friendly model/provider names are a presentation concern.

Inspect:

```
apps/dashboard/src/lib/presentation-labels.ts
```

before renaming canonical IDs.

This allows:

```
router-gpt-5.5
```

to remain the stable ID while:

```
GPT-5.5
```

is shown to a reader.

Presentation precision

Dashboard numeric rendering has a maximum of four fractional digits. The shared formatting layer truncates toward zero when a value exceeds that ceiling rather than rounding it.

Relevant helpers are in:

```
apps/dashboard/src/lib/format.ts
```

including the shared maximum-fraction constant, truncated number/currency formatters, and structured display sanitization used for numeric leaves in structured evidence.

This contract is presentation-only. Do not reduce precision in canonical database rows, generated reviewed evidence, retained source documents, hashes, IDs, timestamps, or model identifiers to make the UI match. Null/unavailable evidence must also remain unavailable rather than becoming numeric zero.

Freshness

Dynamic operational pages can expose execution timestamps and freshness context.

Reviewed snapshots are not "stale" merely because a newer database row exists.

Snapshot date and operational freshness answer different questions.

Do not apply one generic freshness policy to both.

Validity

Invalid/quarantined runs are retained for audit.

Valid-only database views exclude them from scored comparisons.

The system therefore distinguishes:

```
excluded from comparison
```

from:

```
deleted
```

Preserving invalid evidence is important for infrastructure and methodology analysis.

Contamination controls

The benchmark deliberately restricts tools such as WebSearch/WebFetch for relevant arms.

Arm config can pass `disallowed_tools` into the Claude Code harness.

The repository also has contamination audit tooling.

Do not remove a disallowed-tool rule just because a newer harness supports a feature.

First decide whether allowing the tool changes the benchmark condition.

Security boundaries

Important security mechanisms include:

- ignored `.secrets/`;
- runtime-only secret materialization;
- environment clearing;
- command redaction;
- live-output redaction;
- server-side database credentials;
- workspace path containment;
- artifact allowlists;
- secret scanning.

These are layered protections.

No one layer makes the others unnecessary.

Why `.secrets` is different from arm configuration

Arm configuration documents which secret **name** or environment mapping is needed.

`.secrets` contains the actual credential value.

That separation makes arm definitions reviewable and commit-safe.

`make check` as a contract suite

The top-level:

```
make check
```

runs several classes of validation.

At the time of this guide it includes:

- script syntax/compile checks;
- dashboard model/component tests;
- Python test suite;
- comprehensive-review output scan.

Separately:

```
make secret-scan
```

checks for likely leaked secrets.

Before a commit, also use:

```
git diff --check
```

These checks are part of repository change discipline.

Tests are executable documentation

Some of the best explanations of repository invariants are in tests.

Examples:

Current reviewed cost

```
apps/dashboard/src/lib/phase3-current-reviewed-comparison.test.mjs
```

Answers:

- exact scope totals;
- OpenAI provider-billed values;
- allocation firewall;
- historical preservation.

Cost/performance chart

```
apps/dashboard/src/lib/cost-performance-chart.test.mjs
```

Answers:

- exact reviewed run selection;
- metric eligibility;
- Pareto semantics;
- provider filtering;
- qualification display.

Failure composition

```
apps/dashboard/src/lib/failure-composition.test.mjs
```

Answers:

- exact raw-failure population;
- category counts;
- precedence;
- successful-timeout exclusion.

Spend decomposition

```
apps/dashboard/src/lib/spend-decomposition.test.mjs
```

Answers:

- historical source membership;
- outcome buckets;
- accounting gap;
- Kimi historical allocation limitations.

Review loader

```
apps/dashboard/src/lib/review-data.test.mjs
```

and related tests answer:

- manifest validation;
- mixed-output rejection;
- snapshot boundaries.

Publication

Python tests around:

```
canonical_publication
publication_fingerprint
publication_eligibility
live_verification
live_db
path_safety
phase3_freeze
```

are the best place to inspect before changing final publication behavior.

Where do I change X?

Add a new model/backend arm

Start with:

```
configs/arms/<new-arm>.yaml
```

Then inspect:

```
configs/router/litellm.config.yaml.example
scripts/lib/harbor.py
scripts/ensure_arm_runtime_services.sh
.github/workflows/phase3-arm-dispatch-v2.yml
```

Questions:

1. Is the provider already supported by runtime secret mapping?
2. Does LiteLLM need a new route?
3. Does the arm need an auxiliary service?
4. Which environment variables must be cleared?
5. What model alias will Claude Code observe?

6. What backend model should provider evidence identify?
7. Which tools are disallowed?
8. Is this a new phase rather than a legitimate addition to an old frozen phase?

Do not put a new experimental full sweep into closed Phase 3 simply because the workflow can technically run it.

Add a new benchmark phase

Prefer:

- a new phase config;
- new suite identity;
- phase-specific result path;
- explicit reviewed population;
- additive reporting layers.

Inspect:

```
configs/phases/
configs/tasks/
scripts/lib/arms.py
scripts/lib/harbor.py
database suite/arm-run schema
publication policies
dashboard scope registry
reporting generators
```

A new phase should not silently reuse a closed Phase 3 scientific identity.

Add a dashboard-only metric

First classify the intended source:

- operational DB;
- frozen historical snapshot;
- current reviewed snapshot;
- live state.

Then implement:

1. source loader/query;
2. pure model/derivation;
3. explicit unavailable state;
4. page/component;
5. tests;
6. provenance/source label.

Avoid deriving a new metric directly inside JSX if it has methodological meaning.

Change a display label

Prefer presentation-only code such as:

```
apps/dashboard/src/lib/presentation-labels.ts
```

Do not rename:

- arm ID;
- run label;
- trial ID;
- suite ID;
- provider reconciliation key

unless the canonical identity itself truly changes.

Change a cost interpretation

Do not edit an old reviewed snapshot.

Preferred pattern:

1. preserve historical evidence;
2. add a new sanitized input;
3. create a generator for a new reviewed layer;
4. pin source hashes;
5. encode allocation limitations;
6. create/update dashboard loader;
7. add fail-closed tests;
8. migrate only the pages whose research question needs the new basis.

DR-304 is the reference pattern.

Add a failure/trajectory diagnosis

Do not manually edit generated trial classifications.

Preferred pattern:

1. define/extend taxonomy contract;
2. preserve independent axes;
3. update offline classifier;
4. generate new output;
5. bind output to manifest/source hashes;
6. validate exact trial population;
7. expose via loader;
8. update presentation;
9. add targeted evidence tests.

If the new diagnosis depends on absence, require sufficient evidence to justify absence.

Change DR-302 failure composition

Remember that DR-302 is a display partition.

A category/precedence change must:

- preserve the raw-failure denominator;
- be deterministic;

- remain mutually exclusive;
- preserve J2 source data;
- explicitly handle residual unknown/incomplete evidence;
- keep success/not-recorded outside the raw-failure stack.

Change historical DR-303 spend

Treat this as a historical-model change.

Do not feed current provider-billed totals into historical outcome allocation.

If the research question needs current outcome allocation, obtain evidence that actually supports current allocation.

Change current DR-304 cost

Inspect:

```
scripts/generate_phase3_current_reviewed_comparison.py
results/phase3/provider_usage/normalized/
results/phase3/reporting/phase3_current_reviewed_comparison_20260821.json
apps/dashboard/src/lib/phase3-current-reviewed-comparison.ts
```

Do not edit only the generated JSON or TypeScript.

Update the provenance chain.

Add or change an artifact type

Inspect:

```
scripts/ingest_phase3_run_metadata.py
scripts/lib/live_artifacts.py
scripts/lib/path_safety.py
apps/dashboard/src/lib/artifact-types.ts
apps/dashboard/src/lib/artifact-content.ts
artifact presentation components
```

Questions:

- Is the filename allowlisted?
- Is it trial-level or run-level?
- Can it contain secrets?
- Is bounded preview appropriate?
- Does it need content-specific redaction?
- Does progressive publication support it?
- Is its absence expected or anomalous?

Change live supervision

Inspect:

```
scripts/run_arm_live.py
scripts/lib/live_events.py
scripts/lib/live_db.py
scripts/lib/live_artifacts.py
```

```
scripts/lib/live_supervision.py
db/migrations/phase3/009_live_run_supervision.sql
apps/dashboard/src/app/runs/live/
apps/dashboard/src/lib/live-*
```

Preserve:

- benchmark process authority;
- local-first evidence;
- failure isolation;
- redaction;
- bounded shared output;
- mutable/non-canonical status.

For a schema change, add a new migration.

Do not rewrite migration 009.

Change canonical publication

This is one of the highest-risk code areas.

Read, at minimum:

```
scripts/publish_phase3_run.py
scripts/ingest_phase3_run_metadata.py
scripts/lib/publication_eligibility.py
scripts/lib/canonical_publication.py
scripts/lib/live_verification.py
scripts/lib/publication_fingerprint.py
scripts/lib/path_safety.py
scripts/lib/phase3_freeze.py
```

Preserve:

- exact execution discovery;
- structural eligibility;
- local hash verification;
- R2 verification before canonical commit;
- transaction rollback;
- replay safety;
- closed-suite firewall.

Change publication fingerprint

Treat this as a schema/identity decision.

Ask:

- Should old and new manifests representing the same benchmark evidence still replay as identical?
- Is the field scientific identity or supplemental provenance?
- Will old completed publications fail replay after the change?
- Is a fingerprint version bump required?

Do not add fields because "more hashing must be safer."

Identity stability also matters.

Change the database schema

Add a migration under:

```
db/migrations/phase3/
```

or the appropriate future-phase migration location.

Do not alter an already-applied migration to pretend the schema always looked that way.

Update:

- writers;
- dashboard readers;
- verification;
- tests;
- Data Model documentation.

Change runner topology

Runner topology is mostly operational rather than scientific.

Inspect:

```
.github/workflows/phase3-arm-dispatch-v2.yml
Makefile
scripts/eval_remote_ops.sh
runner service configuration outside the repository
```

Keep runner identity/provenance recorded, but do not make one temporary VPS hostname part of model identity.

Implement protected dashboard dispatch later

This is intentionally deferred.

The current Planner should remain review-first until a protected server-side dispatch capability is deliberately implemented.

Future implementation should preserve:

- server-side GitHub credentials;
- arm/mode allowlists;
- dry-run-first UX;
- explicit paid-run authorization;
- runner/provider concurrency validation;
- audit trail;
- workflow execution link;
- publication/ingestion status;
- no browser-side secret.

This remains future platform work and is not part of the completed handoff.

Debugging playbook

Debugging: "The benchmark ran the wrong model"

Read in this order:

1. arm YAML;
2. router config;
3. generated Harbor command;
4. runtime environment;
5. LiteLLM log;
6. Claude Code system/init model evidence;
7. provider evidence where retained.

Distinguish:

- configured alias;
- observed Claude Code model alias;
- routed backend model;
- provider-native model identity.

Do not assume these strings should always be identical.

Debugging: "The pass rate changed"

First determine whether the population changed.

Check:

- reviewed core versus extended;
- valid-imported versus all-imported;
- exact reviewed run versus all imported runs;
- invalid/quarantined inclusion;
- suite/mode;
- attempt count.

Then check raw trial rewards.

Do not begin by assuming scoring changed.

Debugging: "The cost changed"

Identify the cost layer:

- recorded trial cost;
- historical harness total;
- historical reviewed adjusted cost;
- current provider-billed total;
- qualified retained-rate estimate;
- dynamic operational cost.

Then inspect:

- missing cost rows;

- unresolved rows;
- provider reconciliation;
- trial allocation;
- outcome allocation.

Do not fix a cost discrepancy by forcing two different provenance layers to match.

Debugging: "An artifact is missing"

Separate:

1. canonical metadata row missing;
2. R2 URI missing;
3. R2 object unavailable;
4. preview unavailable;
5. object incomplete/bounded;
6. optional artifact never produced.

Inspect:

- trial result directory;
- ingestion manifest;
- artifact database row;
- R2 provenance;
- artifact detail page.

Missing artifact evidence does not always make the raw reward invalid.

Debugging: "Live state says one thing and canonical state says another"

Live state is provisional.

Check:

- live-run status;
- latest heartbeat;
- completed live trials;
- publication state;
- canonical arm-run link;
- final canonical run/trials.

Final canonical publication is the stronger completed evidence layer.

Do not repair canonical data by copying a mutable live counter.

Debugging: "Publication failed"

Read:

```
.run/publish/<id>.json
.run/publish/<id>.log
```

Then classify the stage:

- discovery failed;
- unsafe path/context;
- ineligible;
- missing environment;
- local artifact integrity;
- existing publication mismatch;
- R2 verification;
- database insertion;
- canonical verification;
- closed-suite refusal.

Do not immediately rerun with repair authorization.

Understand the failure first.

Debugging: "A reviewed snapshot disappeared"

Check loader state:

- unavailable;
- stale;
- mixed output.

Then inspect:

- manifest;
- expected output files;
- hashes;
- sizes;
- source hashes;
- row counts.

Do not bypass snapshot validation to make the page render.

Dangerous simplifications

Dangerous simplification 1 — "Use the latest run everywhere"

This destroys reviewed reproducibility.

Reviewed comparisons intentionally pin exact runs.

Dangerous simplification 2 — "Use one cost field everywhere"

Recorded, historical reviewed, provider-billed, and qualified estimated costs have different provenance.

Dangerous simplification 3 — "Failures need one reason"

Raw outcome, response path, verifier failure, policy, termination, and trajectory are independent dimensions.

Dangerous simplification 4 — "R2 URI means verified"

A reference to an object is not a successful content read or integrity check.

Dangerous simplification 5 — "Live and canonical can use one table"

Live state is mutable and partial.

Canonical state is published after final verification.

Dangerous simplification 6 — "Regenerate old outputs with new code"

That destroys the ability to reproduce what an earlier reviewed layer meant.

Prefer a new dated/versioned reviewed layer.

Dangerous simplification 7 — "Remove apparently duplicate tests"

Many tests encode provenance constraints that are not obvious from UI behavior.

Read the assertion before deleting the test.

Dangerous simplification 8 — "Refactor before finding insights"

The incoming team should first use the dashboard to identify valuable research questions.

Code changes should be driven by:

- a discovered limitation;
- a reproducibility issue;
- a high-value analysis need;
- a future experimental phase.

Complexity by itself is not sufficient justification.

Suggested code-reading sequence

A new engineer can gain a useful mental model in this order.

1. Experimental configuration

Read:

```
configs/phases/phase3-router.yaml
configs/arms/router-gpt-5.5.yaml
configs/tasks/terminal-bench-20.txt
```

Goal:

Understand what an arm and phase mean.

2. Command construction

Read:

```
scripts/run_arm.sh
scripts/lib/arms.py
scripts/lib/harbor.py
```

Goal:

Understand exactly what Harbor receives.

3. GitHub workflow

Read:

```
.github/workflows/phase3-arm-dispatch-v2.yml
```

Goal:

Understand paid-run gates, runner selection, services, secrets, live controls, and final publication.

4. Raw result ingestion

Read:

```
scripts/ingest_phase3_run_metadata.py
```

Goal:

Understand how Harbor files become manifest rows and artifacts.

5. Live supervision

Read:

```
scripts/run_arm_live.py
scripts/lib/live_events.py
scripts/lib/live_db.py
scripts/lib/live_supervision.py
scripts/lib/live_artifacts.py
```

Goal:

Understand provisional observation.

6. Final publication

Read:

```
scripts/publish_phase3_run.py
scripts/lib/publication_eligibility.py
scripts/lib/canonical_publication.py
scripts/lib/live_verification.py
scripts/lib/publication_fingerprint.py
scripts/lib/path_safety.py
scripts/lib/phase3_freeze.py
```

Goal:

Understand why completed canonical state is trustworthy.

7. Database schema

Read migrations in order:

```
db/migrations/phase3/001_benchmark_metadata.sql
...
db/migrations/phase3/009_live_run_supervision.sql
db/migrations/phase3/010_cost_authority_semantics.sql
db/migrations/phase3/011_provider_evidence_contract.sql
```

Goal:

Understand schema evolution rather than only the final shape, including raw cost authority, normalized provider evidence/reconciliation, and durable promotion history.

8. Provider evidence and promotion review

Read:

```
scripts/provider_evidence/capability.py
scripts/provider_evidence/providers/anthropic.py
scripts/collect_provider_evidence.py
scripts/lib/evidence_qualification.py
scripts/review_evidence_promotion.py
docs/methodology/USAGE_AND_COST_EVIDENCE_MODEL.md
docs/runbooks/EVIDENCE_PROMOTION_REVIEW.md
```

Goal:

Understand the separation among provider collection, normalized evidence, reviewed reconciliation authority, promotion authorization, and Planner consumption.

9. Operational dashboard data

Read:

```
apps/dashboard/src/lib/db.ts
apps/dashboard/src/lib/dashboard-data.ts
```

Goal:

Understand dynamic populations and SQL sources.

10. Reviewed evidence

Read:

```
apps/dashboard/src/lib/review-data.ts
apps/dashboard/src/lib/failure-taxonomy-snapshot.ts
apps/dashboard/src/lib/phase3-reviewed-comparison.ts
apps/dashboard/src/lib/phase3-current-reviewed-comparison.ts
apps/dashboard/src/lib/phase3-reviewed-run-selection.ts
```

Goal:

Understand frozen versus current-reviewed provenance.

11. Presentation models

Read:

```
apps/dashboard/src/lib/failure-composition.ts
apps/dashboard/src/lib/spend-decomposition.ts
apps/dashboard/src/lib/cost-performance-chart.ts
apps/dashboard/src/lib/cross-phase-reporting.ts
```

Goal:

Understand how research views are derived.

12. Dashboard pages

Read pages only after their data/model layer is understood.

Goal:

See how provenance is communicated to a human.

Relationship between the Codebase Guide and Dashboard Research Guide

Use:

```
docs/guides/DASHBOARD_RESEARCH_GUIDE.md
```

to answer:

> What research question should I ask, and how do I investigate it?

Use this guide to answer:

> Why does the dashboard behave this way, and which code enforces it?

The intended handoff sequence is normally:

```
dashboard question
  -> interesting finding
  -> evidence drilldown
  -> targeted code reading
  -> confidence / limitation
  -> only then a change proposal
```

not:

```
repository clone
  -> broad refactor
  -> hope the research meaning is preserved
```

Questions to answer before changing code

For any substantive change, write down:

1. What research or operational problem does this solve?
2. Which evidence layer does it affect?
3. Is the source raw, canonical, reviewed, current-reviewed, or live?

4. Does the change alter a denominator?
5. Does it alter canonical identity?
6. Does it alter cost provenance?
7. Does it alter failure/trajectory semantics?
8. Does it alter historical reproducibility?
9. Does it require a new migration?
10. Does it require a new reviewed snapshot rather than editing an old one?
11. Which tests encode the current contract?
12. Could this expose a secret?
13. Could this make a partial/live record look canonical?
14. Could this create a new paid benchmark condition?
15. Does the dashboard documentation need updating?

If these questions cannot yet be answered, the code is probably not ready to change.

Repository map by responsibility

Experimental design

```
configs/phases/
configs/arms/
configs/tasks/
configs/router/
```

Benchmark execution

```
scripts/run_arm.sh
scripts/lib/arms.py
scripts/lib/harbor.py
scripts/ensure_litellm_proxy.sh
scripts/ensure_arm_runtime_services.sh
```

Workflow

```
.github/workflows/phase3-arm-dispatch-v2.yml
```

Live observation

```
scripts/run_arm_live.py
scripts/lib/live_events.py
scripts/lib/live_db.py
scripts/lib/live_supervision.py
scripts/lib/live_artifacts.py
```

Canonical publication

```
scripts/publish_phase3_run.py
scripts/ingest_phase3_run_metadata.py
```

```
scripts/lib/canonical_publication.py
scripts/lib/publication_eligibility.py
scripts/lib/publication_fingerprint.py
scripts/lib/live_verification.py
scripts/lib/path_safety.py
scripts/lib/phase3_freeze.py
scripts/lib/runtime_versions.py
```

Database

```
db/migrations/phase3/
```

Offline review/reporting

```
scripts/generate_comprehensive_evidence_review.py
scripts/generate_phase3_current_reviewed_comparison.py
results/manual_verification/
results/phase3/reporting/
docs/reports/phase3/
```

Dashboard data/models

```
apps/dashboard/src/lib/
```

Dashboard UI

```
apps/dashboard/src/app/
apps/dashboard/src/components/
```

Generated dashboard evidence

```
apps/dashboard/src/generated/
```

Validation

```
Makefile
scripts/check.sh
scripts/secret_scan.sh
tests/
apps/dashboard/src/**/*.test.mjs
```

What should remain frozen

Historical research evidence should not be casually regenerated or rewritten.

At minimum, preserve the established frozen/result boundaries documented by the project handoff plan, including:

- Phase 1 results;
- Phase 2 results;
- accepted Phase 3 scored results;

- accepted reviewed snapshots;
- manual-verification snapshots;
- historical DR-302/DR-303 source semantics.

New interpretations should normally be additive.

How future phases should use this architecture

The current architecture provides reusable components:

- config-driven arms;
- task-set selection;
- GitHub Actions safety gates;
- runner slots;
- Harbor execution;
- live observation;
- final publication;
- Supabase/R2 evidence;
- reviewed-snapshot machinery;
- dashboard drilldown.

A future phase should reuse those components where they remain methodologically appropriate.

It should not reuse an old phase identity merely because the infrastructure is convenient.

Phase 4 implication

Phase 4 changes the coding-agent harness.

That means code currently assuming:

```
agent = claude-code
```

must be audited for whether it is:

- genuinely generic;
- only presentation text;
- artifact-name specific;
- Claude Code transcript specific;
- routing specific.

The Phase 4 implementation should therefore begin with a compatibility review, not a blind arm-config expansion.

Phase 5 implication

Phase 5 changes procedure:

```
plan pass
  -> retained plan
  -> execution pass
```

That affects more than model routing.

It creates new evidence and normalization questions:

- planner trajectory;
- plan artifact;
- execution prompt construction;
- two-pass cost;
- two-pass latency;
- failure attribution.

This is why Phase 5 remains methodologically after Phase 4.

Final engineering principle

The repository is strongest when code changes preserve the chain:

```
question
  -> experimental condition
  -> raw evidence
  -> canonical provenance
  -> reviewed interpretation
  -> transparent presentation
```

A change that makes the code cleaner but makes that chain harder to explain is not automatically an improvement.

A change that makes a valuable research question easier to answer while preserving provenance is usually the more important improvement.